Автономная некоммерческая организация высшего образования
«Университет Иннополис»
(АНО ВО «Университет Иннополис»)

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(ДИПЛОМ)
по направлению подготовки
09.03.01 – «Информатика и вычислительная техника»

GRADUATION THESIS
(BATCHLORS GRADUATE THESIS)
Field of Study
09.03.01 – “Computer Science”

Направленность (профиль) образовательной программы
«Анализ данных и искусственный интеллект»
Area of Specialization / Academic Program Title:
“Data Analysis and Artificial Intelligence”

Тема / Topic	**Генеративная адаптация веб-интерфейсов под цели пользователя в режиме реального времени / User-Aligned Web Interfaces: Just-in-Time Generative Adaptation of Existing UIs**

Работу выполнил / Thesis is executed by	**Левада Андрей Романович / Andrey Levada**	подпись / signature
Руководитель выпускной квалификационной работы / Graduation Thesis Supervisor	**Лукманов Рустам Абубакирович / Rustam Lukmanov**	подпись / signature

Иннополис, Innopolis, 2026

Contents

List of Figures

List of Tables

Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Chapter 1

Introduction

1.1 Background

We build User Interfaces to make interactions with tech simpler, easier. Designers generally strive to make the ui as usable as possible for each user (**WIP**: citation needed), but here come 2 problems:

- intrinsically, designers can not feasibly plan interfaces for every single user's specific needs and jobs. with constrained resources, even good interfaces are usually not perfect for each user. For example
- on the other more bleak side are the. Gray and dark patterns systematically misalign system and user goals. This happens when business and user goal diverge, and interface ends up being less helpful and more intrusive (**WIP**: citation needed).

These 2 problems make interfaces misaligned with the users needs — undermining the core of design practice. This is the problem we begin to tackle with this thesis.

Prior HCI research extensivly shows that user interfaces can be made adaptive and customizable. (**WIP**: A bunch of citations needed) These approaches requre

the developer/designer to implement them. As we see in the industry, this is understandably hardly ever a priority (**WIP**: citation needed) except for select adaptivity dimentions — ios and android apps commonly react to the dynamic type accesability settings by changing the layout of the app to better display the content with larger text. Or how websites are almost always made responsive to screen sizes. But as a way of mitigating misaligned interfaces, the adaptive-interface approach is not really used.

Few works take a diffrent approach and try to make interfaces malleable by end users. In this thesis we build upon the ideas they introduce. We want to implement into really this vision of individually tailored user interfaces, shared by many UX practitioners, made possible my recient advaces in LLM's capabilities.

1.2 Internet Shaper

(**WIP**: Shaper demo image)

We design, build and evaluate Internet Shaper — an agentic system wrapped in a browser extension that can change web pages from natural language requests. It works directly on the page in the users browser and does not need access to source code of the website. These changes are persistent across sessions and are powerfull enough to restyle, hide, change elements and while layouts or even bring new, althogh limited, functionality to the website

On evaluation we show how this system enables user-controlled adaptations of existing web interfaces and can function as a way for users to activly realign UIs they use with their needs and interests.

Our contributions are as follows:

1. Internet Shaper as a whole and it's two critical components: a DOM compression algorithm for percieving the web pages; and the Rules Engine for applying chnages to the webpage in a percistent way
2. A pipeline that can create datasets with user-sided natural language edit requests grounded in user personas and jobs
3. And evaluation of an Internet shaper on a dataset gathered form that pipeline

Chapter 2

Related work

There is a longstanding interest of the HCI field practitionerrs in making UIs better by adapting them to the user or giving the user ways to customuze interface for them. Theare have been a number of prominent papers…

They all however are limited by the design-time application. The the devs don’t include the method there is nothing users can do

For papers that are user-side, so propose building the interface for the user’s task (maluable ui paper)

There are a lot less papers that explore the idea of an end-user interface adpatation / customization. These are the closest to our thesis — give the users tools to adapt the UI.

Curent method are however extremely limited — chabges are not percistend, rearly apply to logic. Like these papers we use a browser extension wrapper and read DOM — we then improve processing

Chapter 3

Implementation

(**Section status: Very nice and accurate!**)

This section describes how we designed and built the agentic system at the core of Internet Shaper. We begin from the user-side adaptation problem, explain why a naive read-and-edit baseline is not sufficient, and present the perception and action components that address those failures. The perception and action components are implemented as LLM tools plus a rules engine; the browser extension provides capture, storage, and execution. Source code is listed in the appendix.

3.1 Design Goals

As introduced above, interfaces are rarely perfect for every user. Even well-designed products leave gaps between a person's actual tasks and what the UI optimizes for. In the worst cases, gray and dark patterns deliberately misalign business incentives with user goals, making the interface harder to use for the tasks the user actually cares about.

Most adaptive interface research assumes the application or its developers implement customization hooks. Our approach is different: it is *user-sided*. The user adapts software they already use, without any cooperation from the people who built it. The system operates on pages that were not designed to be modified.

To reach those pages, we host the prototype in a browser extension. Extensions are uniqly positioned to read and write the DOM of any open tab. The page's structure and content are available to the system as HTML — the same representation the browser uses to render what the user sees. This gives us freedom that is miles ahead of other platforms.

Any adaptation system that chnages the UI does 2 sequential steps conceptually. First, it must somehow *read* the page: capture context, reason about structure, or even decompose the user's request into concrete targets. Second, it must *apply* changes to transform the paged into a desired state. These steps are both minimal and required. Throughout this paper we call these two parts *perception* and *action*.

3.2 Baseline Limitations

The straightforward design is to read the full page HTML into the model, then apply direct text or DOM edits. Two practical limits make this baseline infeasible for real applications.

DOM snapshots from real websites are far larger than model context windows allow. We measured token counts on 73 page snapshots from popular domains (corpus collection is described in Section 4.1). Even after removing invisible subtrees,

median visible DOM size remains above 100k tokens; raw snapshots reach 240k tokens at the median and over one million at the maximum.

TABLE 3.1
DOM snapshot sizes — Approximate token counts over 73 page snapshots from the top 25 traffic domains; counted with tiktoken `o200k_base`.

Stage	Mean (tok)	Median (tok)	Max (tok)
Raw DOM	339,253	240,918	1,338,122
Visible DOM	162,806	107,851	1,028,594

It is widly knows that over-saturated contexts produce worse results in general task performance on LLMs (**WIP**: citation needed). Passing the full DOM on every request is therefore not viable: many pages would not fit, and those that do consume most of the context budget before any reasoning begins.

Even when a DOM fits, most of its HTML carries no information relevant to a given adaptation request — build-tool comments, framework wrapper elements, repeated list items, and design-system class tokens. A baseline that reads everything forces the model to filter noise on every turn.

On the action side, direct edit instructions do not survive a page reload. Without persistent storage, the system would need to re-run the full read-and-edit pipeline every time the user opens the page, relying on prompt caching or identical model outputs to reproduce the same result. That is slow, costly, and brittle on dynamic single-page applications where content loads after the initial render.

3.3 Internet Shaper

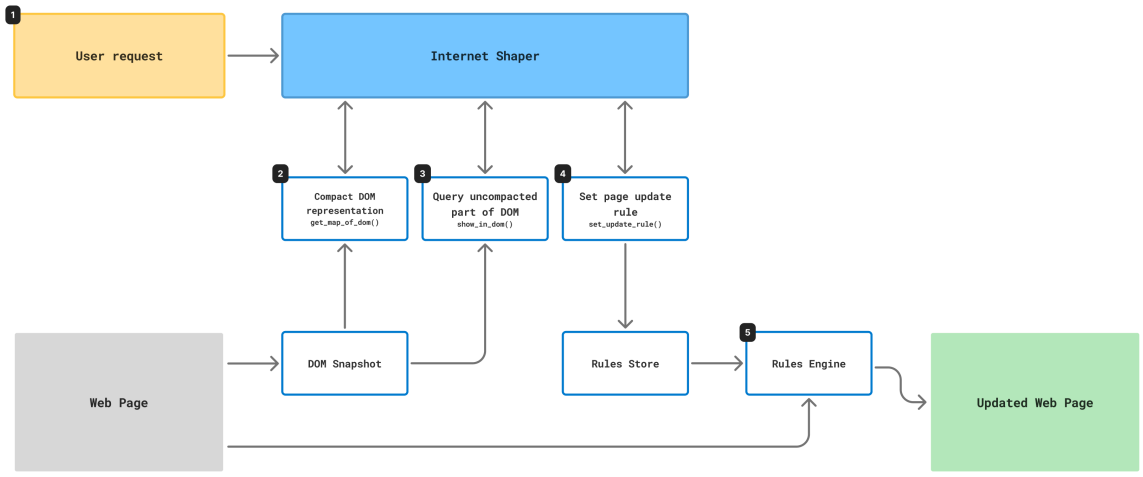

Fig. 3.1. Internet Shaper architecture — Agentic core with browser extension wrapper.

Internet Shaper is an agentic system that makes user-side adaptation of web UIs possible with two specialized components.

Perception is driven by a *DOM Compaction algorithm* gives the model a compact view of the page with the relative structure of elements fully preserved, with a way to retrieve local detail when the overview is not enough.

Action is powered by *Rules Engine*. Instead of direct edits, it records persistent update rules — selector-bound JavaScript that is re-applied on every load and on DOM mutation.

The LLM interacts with both components through tools in a fixed explore-then-act loop. It never writes to the live page directly; it reads from a snapshot frozen at request time and appends rules to a store.

Fig. 3.1 shows the end-to-end workflow inside the browser extension. The numbered steps proceed as follows:

1. The user writes a natural-language adaptation request.
2. The agent calls `get_map_of_dom()` and receives a compact structural map of a DOM snapshot captured at request time.
3. When the map is not enough, the agent calls `show_in_dom()` to inspect local detail from the original snapshot.
4. The agent calls `set_update_rule()` to record persistent, selector-bound JavaScript in the rules store — it does not edit the live page during the loop.
5. After the loop finishes, the rules engine applies stored rules to the live page on load and on DOM mutation, yielding the updated page.

3.4 Perception

An ideal perception interface must meet two criteria that the read-all baseline cannot (Section 3.2):

- *Scale*. The representation must fit within practical context limits even though median visible DOMs already exceed 100k tokens (TABLE 3.1).
- *Signal density*. It must shed the request-irrelevant bulk of production HTML — invisible markup, framework boilerplate, wrapper chains, and repetitive siblings — while preserving enough structure to locate and reason about adaptation targets.

Perception is implemented as a single DOM compression pipeline exposed through two complementary tools on the same captured snapshot: `get_map_of_dom` returns a site-wide structural map; `show_in_dom` retrieves local detail from the uncompressed snapshot when the map is not enough.

3.4.1 DOM Compression Algorithm

The compression pipeline operates on the visible DOM captured at request time (subtrees with `display: none`, `visibility: hidden`, or `opacity: 0` are already removed during capture, as in the evaluation corpus). `get_map_of_dom` applies the following steps in fixed order:

1. *Remove non-structural markup* — drop `head`, `script`, `link`, `style`, and `noscript` elements; build-tool HTML comments; and SVG path data (keeping each `svg` tag and its `title` for accessibility context).
2. *Attribute filtering* — each element keeps only `class`, `id`, `role`, `aria-label`, `label`, `alt`, `type`, `name`, `placeholder`, `value`, and `data-*` attributes; all others (including `href`, `src`, and inline styles) are dropped. Empty attribute values are removed afterward.
3. *High-frequency class removal* — class tokens appearing on more than 5% of elements are stripped tree-wide. The assumption is that very common classes are design-system scaffolding rather than component identifiers.
4. *Single-child wrapper collapse* — chains of elements with exactly one element child and no significant direct text are flattened: intermediate nodes are removed and replaced by an HTML comment recording how many wrappers were collapsed and which class or attribute tokens were lost. Chains stop at leaf elements or at nodes that carry inline text.

```
<body>
  <div id="root">
    <div>
      <div>
        <div>
          <button type="button" title="Go">Go</button>
        </div>
      </div>
    </div>
  </div>
  <ul id="list">
    <li><span>A</span></li>
    <li><span>B</span></li>
    <li><span>C</span></li>
  </ul>
</body>
```

```
<body>
  <div id="root">
    <!-- -3 wrappers -->
    <button type="button" title="Go">Go</button>
  </div>
  <ul id="list">
    <li><span>A</span></li>
    <li><span>B</span></li>
    <li><span>C</span></li>
  </ul>
</body>
```

Fig. 3.2. Single-child wrapper collapse — Before (left) and after (right): three nested `div` wrappers around a button become `<!-- -3 wrappers -->` while the button and unrelated siblings are kept.

5. *Repeated-sibling truncation* — among groups of three or more consecutive siblings deemed structurally similar, only the first is kept; the rest are removed and annotated with a comment giving the truncated count. Similarity requires matching tag name and `id`, approximately matching class tokens (symmetric-difference thresholds scale with class-list length), and approximately matching sequences of immediate child tag names (compared via Levenshtein distance with length-dependent tolerances).

```
<body>
  <div id="root">
    <!-- -3 wrappers -->
    <button type="button" title="Go">Go</button>
  </div>
  <ul id="list">
    <li><span>A</span></li>
    <li><span>B</span></li>
    <li><span>C</span></li>
  </ul>
</body>
```

```
<body>
  <div id="root">
    <!-- -3 wrappers -->
    <button type="button" title="Go">Go</button>
  </div>
  <ul id="list">
    <li><span>A</span></li>
    <!-- -2 siblings -->
  </ul>
</body>
```

Fig. 3.3. Repeated-sibling truncation — Before (left) and after (right): three structurally similar `li` items become one representative plus `<!-- -2 siblings -->`.

6. *Whitespace normalization* — insignificant text nodes are removed and remaining text is collapsed to single spaces, except inside `script`, `style`, `pre`, and `textarea` ancestors.

The result is a compact HTML map that preserves relative hierarchy and enough identifying tokens to locate targets, at the cost of omitting repeated list items and decorative wrapper depth.

3.4.2 Selective Drill-Down

Because compaction is intentionally lossy for scale, the agent can call `show_in_dom(query_selector, depth)` on the *original* snapshot — not the compressed map. The tool resolves the selector against the full captured HTML, clones the matched subtree, and prunes descendants beyond a configurable depth (default three element levels below the matched node). At the depth boundary, nested elements are replaced by a comment of the form `<!-- -N children -->`, while direct text on retained nodes is kept intact.

This complements the map in two ways. First, it restores attributes and child structure removed or summarized during compaction — for example, `href` values, inline styles, or the second item in a truncated sibling group. Second, it bounds local context: the agent requests only the subtree it needs rather than reverting to a full-DOM read. Increasing `depth` trades token cost for completeness when a rule must inspect deep descendants

3.4.3 Alternatives

We also considered alternative comprehension strategies from the web-agent literature. Following [1]‘s classification into text-based, screenshot-based, and multimodal approaches, screenshot-only input is incompatible with our action model, which targets elements via CSS selectors and JavaScript. The accessibility tree preserves semantics for assistive technologies but strips layout detail users often want to change. Task-specific DOM pruning — as in Prune4Web [2] — suits localized edits, but many adaptation requests are global: restyling a feed, suppressing a class of distractions, or restructuring layout. We therefore compress the full visible page structurally rather than pruning to a task-specific subset. We believe a combined aproach of DOM compresssion tigether with pruning might work best, but this is out of scope for this thesis

3.5 Action

An ideal action interface must meet two criteria that one-shot DOM edits cannot (Section 3.2):

- *Persistence*. Changes must survive reload and re-apply on dynamic pages without re-running the full perception pipeline or depending on identical model outputs each visit.
- *Expressiveness*. The mechanism must cover the same edit space direct manipulation allows — styling, interaction changes, and conditional logic over runtime content — not merely the subset CSS can hide or restyle.

Example. "Show only videos that are longer than 40 min" on a YouTube playlist page. A persistent rule must read duration text from each item and conditionally hide non-matching entries — logic that CSS and one-shot text replacements cannot express.

We considered a custom transformation language and generated CSS before settling on JavaScript. Frontier models already generate JS reliably for DOM manipulation, and JS covers the conditional logic CSS cannot. A custom language would add parsing cost without clear benefit.

3.5.1 Rules Engine

Internet Shaper uses update rules. An *update rule* is a persistent, selector-bound transformation stored as a small record: a human-readable `label` (shown in the rule manager UI), a CSS `query_selector`, and a `logic` string of JavaScript. Rules are scoped to the page hostname and persisted in extension storage; on each visit the extension loads the stored set and applies it before the user interacts with the page.

Example. For the YouTube playlist request in the expressiveness example above, the agent might emit a rule like the following:

1. **label:** `Hide videos under 40 min`
2. **query_selector:** `ytd-playlist-video-renderer`
3. **logic:**

```
const duration = element.querySelector('span.ytd-thumbnail-
```

```
    overlay-time-status-renderer')?.textContent?.trim();
    if (!duration) return;
    const parts = duration.split(':').map(Number);
    const minutes = parts.length === 3 ? parts[0] * 60 + parts[1] :
    parts[0];
    if (minutes < 40) element.style.display = 'none';
```

The LLM never executes rule logic during the agent loop. Instead, the `set_update_rule` tool appends a rule to an in-memory list. The tool schema exposes exactly the three fields above and documents the execution contract: the logic runs once per matched element with only an `element` parameter in scope — no `window`, `document`, or other globals. The system prompt further requires idempotent logic (repeated application must converge to the same state), deterministic side effects (set absolute values rather than toggle or append), and graceful handling of not-yet-loaded child content (early return when a value is missing, relying on re-application once content appears). Conditional hiding should prefer `element.style.display = 'none'` over `element.remove()` so the rule can still run when descendants populate.

After the agent loop completes, the new rules are merged into storage and passed to `applyRules`, which injects an applier function into the page's main JavaScript world via the extension's RPC layer. For each enabled rule, the engine queries `document.querySelectorAll`, wraps the logic as `(function(element) { ... })`, and compiles it through a Trusted Types policy where the host page requires one (needed on strict Content Security Policy sites such as YouTube). Each matching element is processed at most once per rule via a per-selector weak set;

a `MutationObserver` on `document.body` re-applies rules to newly inserted nodes, and a short-lived child observer debounces re-runs when descendants of a matched element change — covering single-page applications that hydrate content after the initial render.

From the model's perspective, a rule is therefore a declarative target (`query_selector`) plus imperative body (`logic`); from the runtime's perspective, it is a recurring DOM transformation that survives reloads and dynamic updates without re-invoking the LLM.

3.6 Agent Workflow

The system prompt enforces a strict perception-before-action sequence on every user request. When the agent loop starts, the extension captures a DOM snapshot (`document.documentElement.outerHTML`) and passes it to the tool executor; all perception tools read from this frozen copy rather than the live page. The model then follows this sequence:

1. Call `get_map_of_dom()` before any other tool.
2. Identify candidate elements relevant to the user's request from the returned map.
3. Call `show_in_dom(query_selector, depth)` when selector construction or local structure requires detail absent from the map — for example, distinguishing two elements that collapsed to the same outline, or reading `data-*` values needed for a conditional rule.
4. Emit one or more `set_update_rule(label, query_selector, logic)` calls.

5. Reply to the user when no further tools are needed; the loop continues until the model produces a message without tool calls or the stop reason is `end_turn`, at which point the collected rules are persisted and applied to the live page.

This ordering prevents the model from committing to selectors before it has a structural overview, while still allowing targeted inspection where the compressed map is insufficient. Prompt caching is applied to the system prompt and the `get_map_of_dom` result — typically the largest context item — so multi-turn exploration stays economical. There is also a limit = 1 on the get_map_of_dom calls by the agent to prevent context contamination with duplicate information. We found this is useful during evaluation to prevent agents from trying to get the DOM again to see the review the changes.

3.7 The Browser Extension

The agentic core is hosted in a Chromium browser extension. It captures snapshots, hosts the LLM loop, stores rules in `localStorage`, and applies them in the page's main JavaScript world. Extensions can read and write any tab's DOM without site cooperation, which satisfies the deployment constraint from Section 3.1, but they add no algorithmic behavior beyond capture, RPC, and rule injection.

Browser extensions run code in isolated worlds that cannot call each other directly: content scripts, a background service worker, and the page's main JavaScript context. Fiber, a small framework built for this project, wraps Chrome APIs in an RPC proxy so calling `executeInMainWorld` from a content script behaves like a local function while messages cross process boundaries. Trusted Types policies registered

in the main world allow dynamic compilation of rule logic on strict Content Security Policy sites such as YouTube.

The overlay UI — prompt input, rule manager, status — is implemented with Lit signals in the content script. These concerns are orthogonal to the perception–action design and exist to make the prototype usable.

3.8 Security Concerns

The prototype in its current state has several vulnerabilities and must not be used in a public setting:

- The agentic system has no protection against prompt injections that can be present in page content. This can enable severely harmful behavior up to remote code execution, as the rule application sandbox can fetch data from any URL.
- The browser extension stores API keys in the browser's storage in plain, un-encoded form.

Chapter 4

Methods

This section covers the data collection, processing piplines, evaluation task synthesis, and the evaluation process itself.

4.1 DOM Compression Evaluation

To move from anecdotal DOM sizes to measurable compression, we assembled a corpus of real page snapshots and ran the perception pipeline on each one.

4.1.1 Source and filtering

We started from the top-domain list in the web-unpacked study, which annotates 116 high-traffic sites. We filtered out login- or payment-gated services and adult-industry domains, because manual evaluation might later involve inspecting page screenshots. We kept the top 25 remaining domains by rank.

Several homepages needed manual URL adjustment before crawling: search engines and YouTube were pointed at result pages rather than landing pages, Wikipedia at the English homepage, and Pinterest at a browsable feed. We then

crawled same-domain links from each seed URL using a headed browser, semi-automatically accepting cookie banners and manually passing captchas where required.

Further manual filtering removed dead or region-locked services, pages blocked by bot detection, legal pages (terms of service, privacy policy), regional duplicates, and sitemaps. From each surviving domain we sampled the homepage plus three randomly chosen internal pages.

4.1.2 Capture and quality control

Each selected URL was snapshotted with Playwright. For every page we stored:

1. `raw.html` — the full serialized DOM.
2. `visible.html` — the DOM after removing subtrees that are invisible by computed style (`display: none`, `visibility: hidden`, `opacity: 0`), matching the capture step used in the perception component.
3. A screenshot for optional human review.

Captchas and empty bot-rejection pages were handled in a semi-manual patch pass; snapshots that remained empty or unusable were dropped. The final corpus contains 73 pages across 25 domains.

4.1.3 Compression measurements

We ran the production `get_map_of_dom` pipeline on each snapshot's `visible.html` and counted tokens with tiktoken's `o200k_base` encoding. Across the corpus:

TABLE 4.1
Corpus compression results — Token counts over 73 snapshots; compressed = production `get_map_of_dom` output

Stage	Mean tokens	Median tokens
Raw DOM	339,253	240,918
Visible DOM	162,806	107,851
Compressed map	11,135	6,613

Visible capture alone cuts median size by roughly half; the compressed map reduces it by another order of magnitude. This confirmed that a compact initial view plus selective drill-down is necessary rather than optional.

4.2 Task Synthesis

Compression metrics alone do not define adaptation tasks. To evaluate the full agent, we synthesized user requests from the snapshot corpus using a Jobs-to-be-Done (JTBD) pipeline.

We selected 10 snapshots from the corpus with a fixed random seed, excluding pages that failed quality checks. For each snapshot, a separate LLM session (Gemini 3.5 Flash) inspected the page DOM and produced, in three turns:

1. A list of user jobs on the page ("I want to…").
2. Three pairs of opposing user preferences — persistent traits rather than one-off edits.
3. Six concrete edit requests — one per preference side, labeled `1a` through `3b`.

Each request became a *seed sample*: a `task.json` describing the prompt and JTBD context, plus copies of the snapshot's `raw.html` and `visible.html`. This yields 60

seed tasks grounded in real pages and varied user intents. Downstream ablation pipelines (Section 4.3) run different perception–action combinations on these seeds for controlled comparison.

4.3 Ablation Pipelines

To isolate the perception and action components, we run six agent configurations on the synthesized seed tasks:

TABLE 4.2
Agent ablation conditions

Pipeline	Perception	Action
Baseline	full DOM (`get_dom`)	immediate patch (`edit`)
Engine only	full DOM	persistent rules
Map only	map + drill-down	immediate patch
Full	map + drill-down	persistent rules
Full (Sonnet)	map + drill-down	persistent rules, Claude Sonnet 4.6

Baseline and map-only swap perception strategy while keeping a one-shot edit action. Engine-only and full swap persistence while keeping full-DOM or compact perception respectively. The full pipeline matches the production prototype; results are reported in the Results section.

(**WIP: how the test was run, the pairvise test stand the ststaistical test used for eval**)

4.4 AI Usage Disclosure

AI agents via Cursor Editor and Claude Code CLI were used in writing the prototype code. All generated code was fully reviewed and verified manually, and all architectural decisions were made explicitly by the authors.

Chapter 5

Results

5.1 DOM Compression Algorithm

5.2 Pairwise Preference Evaluation

Chapter 6

Discussion

(**WIP**: why these results, how they are interesting; each вывод is in it's own numbered callout block)

6.1 Emergent Behaviors

Even more complex queries produced unexpectedly positive results, with the agent exhibiting emergent behavior that was not planned for:

- When asked to "add a cat to the page", it called an external API dedicated to serving a random cat image and injected a new image element into the page. > Cat image added — a random cat image (fetched from cataas.com) has been inserted into the center of the page, with rounded corners and a soft shadow.
- For requests more complex than "remove element A", the agent immediately used variables, conditionals, and accessed properties of child nodes from the parent.
- When asked to turn a list of articles on Substack into a grid, it produced a set of approximately seven rules, adapting not only the layout but also the cards themselves — their headers, thumbnails, and action menus — all to better fit the new

grid context. This example shows the agent's high-level understanding of tasks rather than mechanical rule application.

- When asked to restructure the page in a major way, moving sections around, the agent created a single rule for the `main` element containing all the logic, rather than many individual rules.

Chapter 7

Conclusion

(**WIP**: in the end this is what we got and how we a)

7.1 Limitations

1. The LLM is provided a single screen in an application and is not aware of the multi-screen context. The risk is that it might produce adaptations incompatible with the overall flow on more complex tasks.
2. The system can only influence a single screen, making construction of multi-step flows and app-wide modifications difficult.
3. Rules are applied to all pages that match the domain; no URL path matching occurs. As a result, rules sometimes unintentionally apply to pages they were not intended for. This was an architectural decision that helps handle dynamically constructed pages (e.g. `domain.com/user/<id>`). Future work can investigate the feasibility of requiring a URL mask for each rule, or a similar approach to limit unexpected rule applications.

4. The LLM has access only to data visible to the user and present in the DOM. For a subset of interactions — specifically those involving dynamically loaded data — access to the underlying JavaScript data structures could be helpful. The same applies to API actions, of which the model has no context at all.

Bibliography cited

[1] L. Ning *et al.*, “A Survey of WebAgents: Towards Next-Generation AI Agents for Web Automation with Large Foundation Models.” Accessed: Apr. 14, 2026. [Online]. Available: http://arxiv.org/abs/2503.23350

[2] J. Zhang, K. Chen, Z. Lu, E. Zhou, Q. Yu, and J. Zhang, “Prune4Web: DOM Tree Pruning Programming for Web Agent.” Accessed: May 18, 2026. [Online]. Available: https://arxiv.org/abs/2511.21398

Appendix A

Source Code

WIP